

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

«Алгоритмы и структуры данных»

отчет по блоку задач №4 (Яндекс.Контест)

Выполнил:

Студент группы Р3210

Давидюк Антон

Задача №1 М. Цивилизация

Разбор: dist хранит минимальное время достижения каждой клетки, инициализируется INF как предел. from запоминает координаты предыдущей клетки для восстановления маршрута, а lastMove фиксирует направление (N, E, S, W), по которому мы попали в клетку. State и priority_queue с greater реализуют очередь с приоритетом для алгоритма Дейкстры. Массивы deltax, deltax и dirChar задают смещения по осям и буквы ходов. В основном цикле алгоритм извлекает клетку с минимальным временем, пропускает устаревшие состояния, проверяет соседей на выход за границы карты и наличие непроходимой воды (#). При релаксации ребра обновляются dist, from, lastMove, и новое состояние помещается в очередь. После завершения поиска путь восстанавливается обратным проходом от финиша к старту через lastMove и from, после чего строка разворачивается функцией reverse. Проверка dist[end] == INF обрабатывает случай, когда клетка полностью заблокирована.

Алгоритмическая сложность:

1. По времени: Алгоритм Дейкстры на сеточном графе с $V = N \cdot M$ вершинами и $E \approx 4V$ рёбрами. Каждая вершина и ребро обрабатываются не более одного раза с приоритетной очередью, что даёт $O(E \log V) = O(N \cdot M \log(N \cdot M))$. Восстановление пути выполняется за $O(N \cdot M)$ в худшем случае. Итоговая сложность $O(N \cdot M \log(N \cdot M))$.
2. По памяти: Матрицы dist, from и lastMove занимают $O(N \cdot M)$ памяти каждая. Приоритетная очередь в худшем случае может хранить $O(N \cdot M)$ элементов. Вспомогательные структуры используют линейное количество памяти относительно размеров карты. Итоговая сложность $O(N \cdot M)$.

Код:

```
#include <iostream>
#include <vector>
#include <queue>
#include <algorithm>

struct State {
    int r, c, cost;
    State(int row, int col, int total) : r(row), c(col), cost(total) {}

    bool operator>(const State& other) const {
        return cost > other.cost;
    }
};

int main() {

    int N, M;
    std::cin >> N >> M;

    int startR, startC, endR, endC;
    std::cin >> startR >> startC >> endR >> endC;

    startR--; startC--; endR--; endC--;

    std::vector<std::string> grid(N);
    for (int i = 0; i < N; i++) {
        std::cin >> grid[i];
    }

    const int deltax[4] = { -1, 0, 1, 0 };
    const int deltax[4] = { 0, 1, 0, -1 };
```

```

const char dirChar[4] = { 'N', 'E', 'S', 'W' };

const int INF = 1000000000;
std::vector<std::vector<int>> dist(N, std::vector<int>(M, INF));

std::vector<std::vector<std::pair<int, int>>> from(N, std::vector<std::pair<int,
int>>(M, { -1,-1 }));

std::vector<std::vector<char>> lastMove(N, std::vector<char>(M, 0));

std::priority_queue<State, std::vector<State>, std::greater<State>> pq;

dist[startR][startC] = 0;
pq.push(State(startR, startC, 0));

while (!pq.empty()) {
    State cur = pq.top();
    pq.pop();

    if (cur.cost != dist[cur.r][cur.c]) {
        continue;
    }

    if (cur.r == endR && cur.c == endC) {
        break;
    }

    for (int d = 0; d < 4; d++) {
        int nextx = cur.r + deltax[d];
        int nexty = cur.c + deltay[d];

        if (nextx < 0 || nextx >= N || nexty < 0 || nexty >= M) {
            continue;
        }

        if (grid[nextx][nexty] == '#') {
            continue;
        }

        int moveCost = (grid[nextx][nexty] == 'W') ? 2 : 1;
        int newCost = cur.cost + moveCost;

        if (newCost < dist[nextx][nexty]) {
            dist[nextx][nexty] = newCost;
            from[nextx][nexty] = { cur.r, cur.c };
            lastMove[nextx][nexty] = dirChar[d];
            pq.push(State(nextx, nexty, newCost));
        }
    }
}

if (dist[endR][endC] == INF) {
    std::cout << -1 << std::endl;
    return 0;
}

std::string path = "";
int cur_x = endR, cur_y = endC;

while (cur_x != startR || cur_y != startC) {
    path += lastMove[cur_x][cur_y];
    int prev_x = from[cur_x][cur_y].first;
    int prev_y = from[cur_x][cur_y].second;
    cur_x = prev_x;

```

```

        cur_y = prev_y;
    }

    reverse(path.begin(), path.end());

    std::cout << dist[endR][endC] << std::endl;
    std::cout << path << std::endl;

    return 0;
}

```

Задача №2 N. Свинки-копилки

Разбор:

graph хранит список смежности графа, где ребро между i и key_holder означает, что копилки находятся в одной связной компоненте. visited маркирует уже обработанные копилки, предотвращая повторные обходы и заикливание. breaks_needed аккумулирует количество обнаруженных компонент связности. В основном цикле алгоритм последовательно перебирает все вершины от 1 до n . При встрече непосещённой копилки счётчик увеличивается на 1. Запускается итеративный обход в глубину, вектор to_visit работает как стек, вершина извлекается, а все её непосещённые соседи помечаются и добавляются в стек для дальнейшего исследования. Обход продолжается до полного покрытия.

Алгоритмическая сложность:

1. По времени: Построение графа выполняется за $O(N)$, так как для каждой из N копилки добавляется одно ребро. Итеративный обход в глубину посещает каждую вершину и каждое ребро строго один раз. Итоговая сложность $O(N)$.
2. По памяти: Список смежности graph хранит $O(N)$ записей (каждая копилка связана ровно с одной другой в неориентированном представлении). Массив visited и стек to_visit используют линейное количество памяти относительно N . Итоговая сложность $O(N)$.

Код:

```

#include <iostream>
#include <vector>

using namespace std;

int main() {
    int total_piggy_banks;
    cin >> total_piggy_banks;

    vector<vector<int>> graph(total_piggy_banks + 1);
    for (int i = 1; i <= total_piggy_banks; ++i) {
        int key_holder;
        cin >> key_holder;
    }
}

```

```

        graph[i].push_back(key_holder);
        graph[key_holder].push_back(i);
    }

    vector<bool> visited(total_piggy_banks + 1, false);
    int breaks_needed = 0;

    for (int start = 1; start <= total_piggy_banks; ++start) {
        if (visited[start]) continue;

        breaks_needed++;

        vector<int> to_visit;
        to_visit.push_back(start);
        visited[start] = true;

        while (!to_visit.empty()) {
            int current = to_visit.back();
            to_visit.pop_back();

            for (int neighbor : graph[current]) {
                if (!visited[neighbor]) {
                    visited[neighbor] = true;
                    to_visit.push_back(neighbor);
                }
            }
        }
    }

    cout << breaks_needed << std::endl;
    return 0;
}

```

Задача №3 О. Долой списывание!

Разбор: `exchange_graph` хранит список смежности неориентированного графа, где рёбра соответствуют парам. `group_id` показывает принадлежность к группам (0 = не распределён, 1 = первая группа, 2 = вторая группа), предотвращая повторные обработки вершин. `can_split` служит флагом раннего выхода при обнаружении невозможности разбиения. В основном цикле алгоритм последовательно перебирает всех участников от 1 до N. Обход в ширину (BFS) через `traversal_queue`: стартовой вершине назначается группа 1. На каждом шаге извлекается текущий участник, проверяются все его соседи по графу. Если сосед ещё не распределён, ему назначается противоположная группа (вычисляется как 3 - текущая_группа) и он помещается в очередь. Если сосед уже распределён и принадлежит той же группе, что и текущий, условие задачи нарушается (обнаружен нечётный цикл), флаг `can_split` устанавливается в false и обработка прерывается. Если после обхода всех компонент связности конфликтов не выявлено, разбиение на две допустимые группы возможно.

Алгоритмическая сложность:

1. По времени: Построение списка смежности выполняется за $O(M)$. Обход в ширину посещает каждую вершину и каждое ребро строго один раз, операции внутри цикла занимают $O(1)$. Итоговая сложность $O(N + M)$.
2. По памяти: Список смежности `exchange_graph` занимает $O(N + M)$ памяти для хранения всех рёбер. Вектор `group_id` и очередь `traversal_queue` используют $O(N)$ памяти. Вспомогательные структуры не требуют дополнительной асимптотики. Итоговая сложность $O(N + M)$.

Код:

```
#include <iostream>
#include <vector>
#include <queue>

int main() {
    int students_count, exchanges_count;
    std::cin >> students_count >> exchanges_count;

    std::vector<std::vector<int>> exchange_graph(students_count + 1);
    for (int i = 0; i < exchanges_count; ++i) {
        int student_a, student_b;
        std::cin >> student_a >> student_b;
        exchange_graph[student_a].push_back(student_b);
        exchange_graph[student_b].push_back(student_a);
    }

    std::vector<int> group_id(students_count + 1, 0);

    bool can_split = true;

    for (int starter = 1; starter <= students_count && can_split; ++starter) {
        if (group_id[starter] != 0) {
            continue;
        }

        std::queue<int> traversal_queue;
        traversal_queue.push(starter);
        group_id[starter] = 1;

        while (!traversal_queue.empty() && can_split) {
            int current_student = traversal_queue.front();
            traversal_queue.pop();

            for (int partner : exchange_graph[current_student]) {
                if (group_id[partner] == 0) {
                    group_id[partner] = 3 - group_id[current_student];
                    traversal_queue.push(partner);
                }
                else if (group_id[partner] == group_id[current_student]) {
                    can_split = false;
                    break;
                }
            }
        }
    }

    std::cout << (can_split ? "YES" : "NO") << std::endl;
    return 0;
}
```

Задача №4 Р. Авиаперелёты

Разбор:

`route_matrix` хранит полную матрицу смежности с весами перелётов между городами. `tank_thresholds` аккумулирует все уникальные значения рёбер, после сортировки и удаления дубликатов образует монотонную последовательность для бинарного поиска. Лямбда `is_strongly_connected` проверяет сильную связность подграфа, образованного рёбрами с весом не более `max_capacity`: запускает два независимых обхода в ширину (прямой по исходным рёбрам и обратный по транспонированным) из нулевой вершины, подсчитывая количество посещённых узлов. Если оба обхода охватывают все города, ограничение считается допустимым. В основном цикле бинарного поиска при успешной проверке фиксируется кандидат `optimal_tank` и диапазон сдвигается влево, иначе вправо.

Алгоритмическая сложность:

1. По времени: Чтение входных данных и сбор весов выполняются за $O(N^2)$. Сортировка уникальных порогов занимает $O(N^2 \log N)$. Бинарный поиск делает $O(\log N)$ итераций, каждая из которых запускает два обхода в ширину по плотной матрице смежности, что требует $O(N^2)$ операций. Итоговая сложность $O(N^2 \log N)$.
2. По памяти: Матрица `route_matrix` и массив уникальных весов `tank_thresholds` требуют $O(N^2)$ памяти для хранения входных данных. Векторы посещённых вершин и очереди обхода используют линейную память $O(N)$. Вспомогательные структуры не увеличивают асимптотику. Итоговая сложность $O(N^2)$.

Код:

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <queue>

int main() {
    int cities_count;
    std::cin >> cities_count;

    if (cities_count == 1) {
        std::cout << 0 << std::endl;
        return 0;
    }

    std::vector<std::vector<int>> route_matrix(cities_count,
std::vector<int>(cities_count));
    std::vector<int> tank_thresholds;
    tank_thresholds.reserve(cities_count * cities_count);

    for (int i = 0; i < cities_count; ++i) {
        for (int j = 0; j < cities_count; ++j) {
            std::cin >> route_matrix[i][j];
            if (i != j) {
                tank_thresholds.push_back(route_matrix[i][j]);
            }
        }
    }
}
```

```

    }
}

sort(tank_thresholds.begin(), tank_thresholds.end());
tank_thresholds.erase(unique(tank_thresholds.begin(), tank_thresholds.end()),
tank_thresholds.end());

auto is_strongly_connected = [&](int max_capacity) -> bool {
    std::vector<char> visited_fwd(cities_count, 0);
    std::vector<char> visited_rev(cities_count, 0);
    std::queue<int> q_fwd, q_rev;
    int count_fwd = 1, count_rev = 1;

    q_fwd.push(0);
    visited_fwd[0] = 1;
    while (!q_fwd.empty()) {
        int current = q_fwd.front();
        q_fwd.pop();
        for (int next = 0; next < cities_count; ++next) {
            if (!visited_fwd[next] && route_matrix[current][next] <=
max_capacity) {
                visited_fwd[next] = 1;
                count_fwd++;
                q_fwd.push(next);
            }
        }
    }

    q_rev.push(0);
    visited_rev[0] = 1;
    while (!q_rev.empty()) {
        int current = q_rev.front();
        q_rev.pop();
        for (int prev = 0; prev < cities_count; ++prev) {
            if (!visited_rev[prev] && route_matrix[prev][current] <=
max_capacity) {
                visited_rev[prev] = 1;
                count_rev++;
                q_rev.push(prev);
            }
        }
    }

    return count_fwd == cities_count && count_rev == cities_count;
};

int left = 0, right = static_cast<int>(tank_thresholds.size()) - 1;
int optimal_tank = tank_thresholds.back();

while (left <= right) {
    int mid = left + (right - left) / 2;
    if (is_strongly_connected(tank_thresholds[mid])) {
        optimal_tank = tank_thresholds[mid];
        right = mid - 1;
    }
    else {
        left = mid + 1;
    }
}

std::cout << optimal_tank << std::endl;
return 0;
}

```